INDEX

S.No.	Date	Name of the Experiment	Mapping To Course Outcome	Grade	Signature
1	1/7/26	Introduction & Basic HTML	CO1	13	
2	8/7	Design HTML using audio	CO1	14	
3	15/07	Design HTML using video	CO1	14	
4.	22/07	Program using Java Script (functions & arrays)	CO1	13	
5.	29/7	Develop dynamic webpage	CO1	13	
6	5/8	Implement modules in node.js	CO2	14	
7	12/8	mini application using express and nodejs	CO2	14	
8.	19/8	Code tantra - Static webpage	CO1		
9.	2/9	Code tantra - portfolio	CO1		
10.	9/9	Code tantra - Simple calculator	CO1		

INDEX

S.No.	Date	Name of the Experiment	Mapping To Course Outcome	Grade	Signature
11.	16/9	Create a functional and class	CO_2		
12.	23/9	Implement http methods	CO_2		
13.	30/9	Develop CRUD operations	CO_3		

Anil Neerukonda Institute of Technology & Sciences
(Autonomous)
Affiliated to AU, Approved by AICTE & Accredited by NAAC with A+ Grade
Accredited by NBA (B.Tech – ECE, EEE, CSE, IT, MECH, Civil & Chemical)
Sangivalasa-531 162, Bheemunipatnam Mandal, Visakhapatnam District
Phone: 8712005999, 8712008222, Website: www.anits.edu.in, email:

Department of Computer Science & Engineering (AI & ML)

Academic Year:	2026-2027	**Semester:**	I
Name of the Student:	D Jaswanthraju	**Roll Number:**	322126552137
Name of the Lab:	FS Lab	**Timings:**	9:40 am to 12:10 pm

LAB WEEKLY ASSESSMENT SHEET

Date	Experiment /Week No	Observation Book				Total (15 Marks)	Record (10 Marks)	Faculty Remarks
		Theoretical Knowledge (4 Marks)	Programming Knowledge (4 Marks)	Results (5 Marks)	Viva-Voce (2 Marks)			
1/7/26	1	3	3	5	2	13		
8/7/26	2	4	3	5	2	14		
15/7/26	3	4	3	5	2	14		
22/7/26	4	3	4	4	2	13		
29/7/26	5	4	4	4	1	13		
5/8/26	6	3	4	5	1	14		
9/8/26	7	4	3	4	2	13		
26/8/26	8	4	3	4	2	14		
2/9	9	3	3	5	2	13		
9/9	10	4	3	5	2	14		
16/9	11	4	4	5	2	15		
23/9	12	4	4	5	2	15		
30/9	13	4	4	5	2	15		

Name of the Faculty: Swathi

Faculty Signature:

01/7/26

Introduction to HTML and CSS

→ HTML (hyper text markup language) is the standad markup language used to create & structure content on to create a webpage using elements and other multimedia content. HTML document are interpreted by the web brousers to display the content in a structured and meaningful manner. HTML provides the structure & semantic organization of a web page.

→ CSS (cascading style sheet) is a stylesheet language used to control the presentation and visual appearance of HTML documents. It is used to define properties such as colours, fonts, backgrounds, spacing, borders, alignment, dimension and layers. CSS also support responsive design, allowing webpages to adapt to different screen sizes and devices. CSS can be applied using inline, internal & External.

-> HTML and CSS work together to develop webpages where HTML provides the structure & content while CSS provides the styling layout & visual presentation. The combination of both forms the foundation of modern web development & enables the creation of organized, attractive and user-friendly web interfaces.

Difference btw MERS & full stack

-> Mean stack is a javascript based technology stack used for developing modern web application. MERN stand for MongoDB, Express.js, React.js, Node.js.

-> Full stack Development refers to the development of both the frontend and backend of a web application along the database integration. A full stack developer works with technologies used for designing the user interface, implementing server side logic, managing data bases.

8/9/26

HTML Components:

1) HTML Tags - define the structure & elements of a webpage.

2) Heading - used to display titles and heading using <h1> to <h6>

3) Paragraphs - used to display text using <p> tag

4) Links - used to navigate b webpages <a> tag

5) Images - used to display image using <image> tag.

6) Lists - used to create ordered & and unordered lists using <ol>, <ul>, <li>.

7) Tables - used to organise data in rows & columns.

8) Forms - used to collect user i/p elements such as text boxes, buttons & check boxes

9) div & span - used to group and diag organize webpage content

10) Semantic elements are <header>, <nav>, <section>, <article>, <footer>

provide meaningful webpages structure.

CSS Components

Selectors – select HTML elements to apply styles

properties – Define visual characteristics of elements

Values – specify setting of CSS property

Colors – Define text, bg, borders

Font – Control font family, size style & weight.

Background – define the set bg colors, img.

Border – define borders style, width, radius

Margin – Controls space outside element

padding – space inside element.

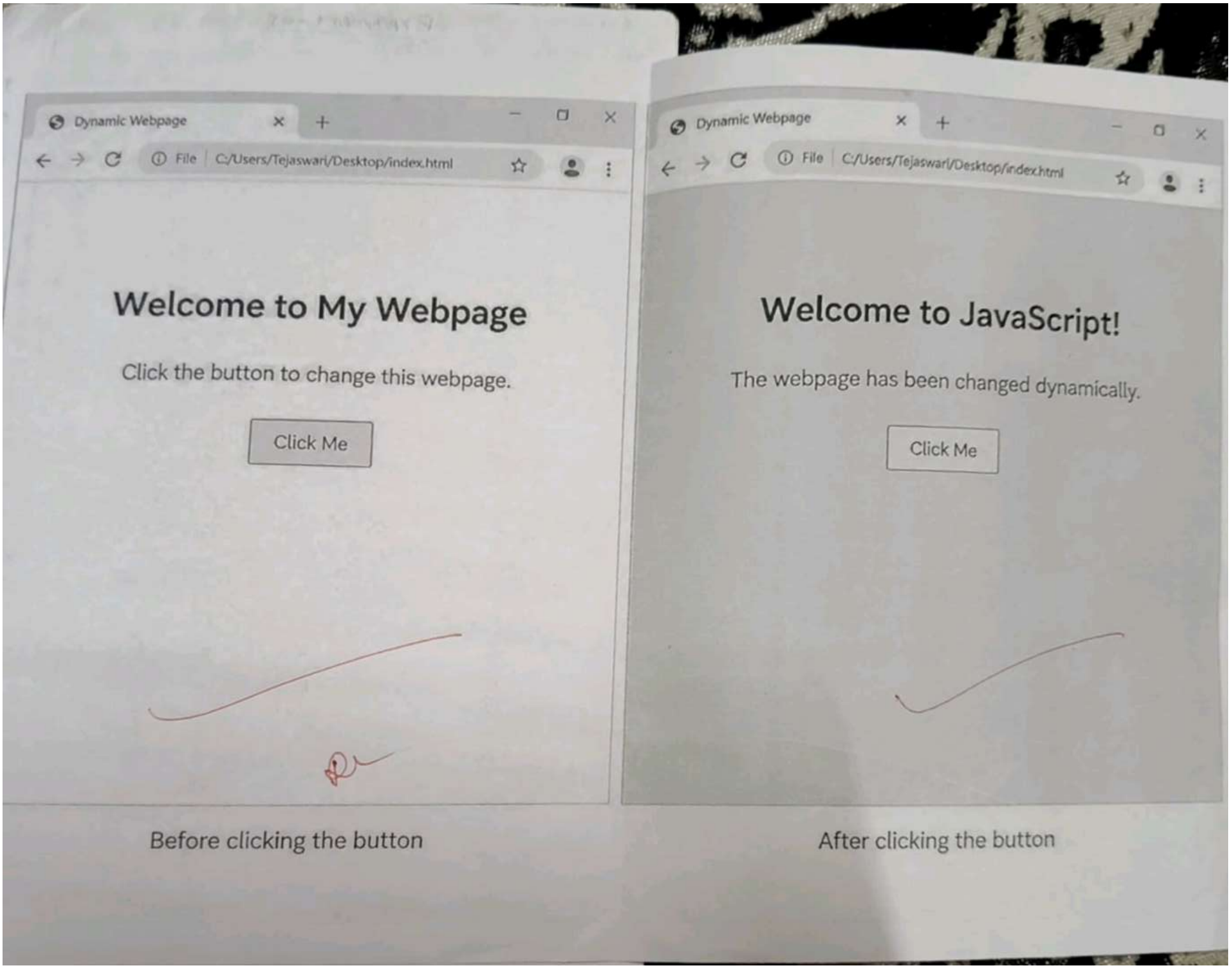
Layout – Control the arrangement of elements using flexbox, grid and positioning.

HTML VS HTML5

HTML	HTML5
1) Older version of HTML	1) latest major version of HTML
2) limited Multimedia support	2) Support audio & video
3) Uses external plugins for multimedia	3) No external plugs needs
4) Has fewer semantic tags	4) Provides semantic tags
5) No <canvas> element	5) Supports <canvas>
6) Limited web storage	6) Support local storage
7) Longer Doctype declaration	7) Simple <!DOCTYPE HTML>

CSS VS CSS3

CSS	CSS3
1) Earlier version of CSS	1) Advanced Version of CSS
2) Basic styling features	2) provides advanced
3) limited animations	3) Supports animations and Transition
4) No flexible layout systems	4) Supports flexbox and grid.
5) limited visual effects	5) Supports shadows, gradients effects
6) No media queries	6) Supports media queries.



Before clicking the button

After clicking the button

innerHTML = "welcome to javascript";

document.getElementById("heading").

innerHTML = "The webpage has been changed dynamically"

document.body.style.backgroundColor = "light"

}

</script>

</body>

</html>

* Output:

welcome to My Webpage

Click the button to change this webpage

[clickme]

→ when the user clicks "click me", the java's function changePage() is called then the heading dynamically changes and background also changes

- After clicking the button:

welcome to JavaScript

The webpage has been changed dynamically

[click me.]



Write a program using JavaScript (arrays, functions)

Code:

```
let numbers = [12, 45, 8, 23, 56, 19, 34, 67, 90, 1];

function displayArray(arr){
    console.log("Array Elements:");

    for (let i=0; i<arr.length; ++i){
        console.log(arr[i]);

    }
}

function findSum(arr){

    let sum=0;
    for (let i=0; i< arr.length; i++){
        sum+= arr[i];

    }
    return sum;
}
```

Experiment-04

i) Develop dynamic webpage using java script

Program:

```
<!DOCTYPE html>
<html>
<head>
    <title> Dynamic webpage </title>
    <style>
        body{
            text-align : center;
            font-family: Arial;
            margin-top : 100px;
            }
        button{
            padding: 10px 20px;
            font-size: 16px;
            cursor : pointer;
            }
    </style>
</head>
<body>
    <h1 id="heading"> Welcome to
        My webpage </h1>
    <p id="message"> click the button
        to change this webpage </p>
    <button onclick="changepage()">
        Click Me </button>

<script>
    function changepage(){
    document.getElementById("heading")
```



22/2/26

i) Design an HTML code using audio tag.

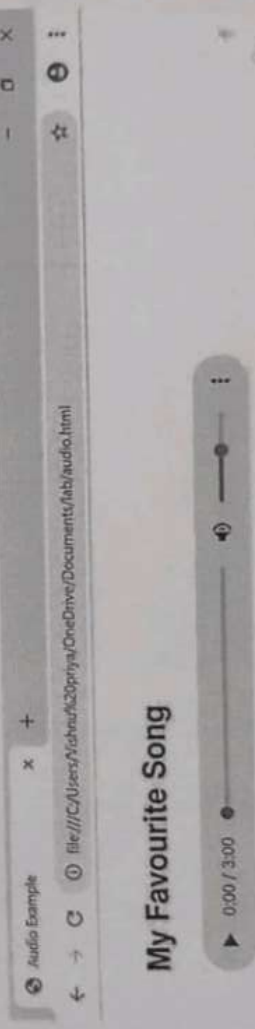
Code:

```
<!DOCTYPE html>
<html>
<head>
    <title> Audio Example</title>
</head>
<body>
    <h2> My Favorite Song</h2>
    <audio controls>
        <source src="song.mp3" type="audio/mpeg">
        your browser does not support the audio
        tag
    </audio>
</body>
</html>
```

ii. Display an audio player with play, pause, and volume controls.

Steps:

Create a new HTML file
Add the basic HTML structure
Add heading using <h2> tag
Insert the <audio> tag with controls attribute.
Specify the audio file using <source> tag
Save and open the HTML file in a web browser
Play the audio using displayed controls



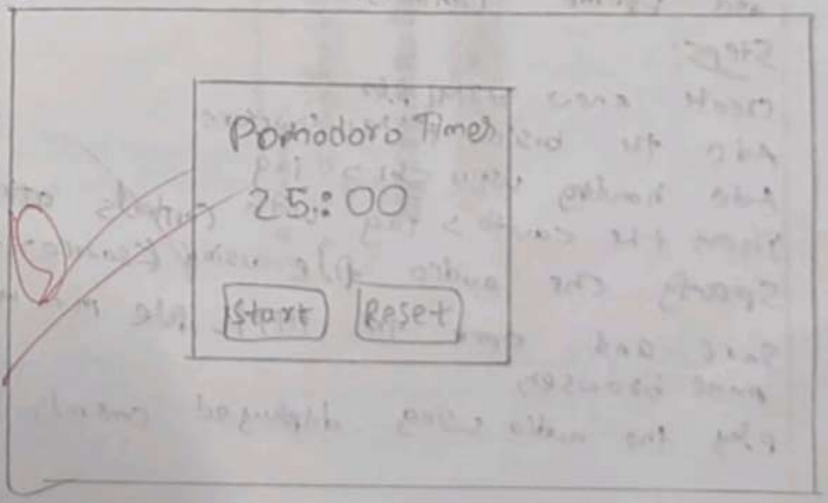
15/9/26

*Design a static web page using HTML and CSS

Pomodoro Timer, using HTML, CSS

```
<!DOCTYPE html>
<html>
<head>
    <title> Pomodoro Timer</title>
    <style>
        body{
            font-family: Arial;
            background: #f5f5f5;
            text-align: center;
            padding: 50px;
        }
        .timer{
            width: 300px;
            margin: auto;
            padding: 30px;
            background: white;
            border-radius: 15px;
            box-shadow: 0 0 10px gray;
        }
        h1{
            color: #e74c3c;
        }
        .time{
            font-size: 50px
            margin: 20px
        }
        button{
            padding: 10px 20px
            margin: 5px
            border: none;
            border-radius: 5px;
            background: #e74c3c;
            color: white;
        }
    </style>
</head>
<body>
    <div class="Timer">
        <h1> Pomodoro Timer</h1>
        <div class="time"> 25:00</div>
        <button> start </button>
        <button> Reset</button>
    </div>
</body>
</html>
```

O/P:



Experiment - 05

Implement modules in node JS

- In node.js, you can implement modules using distinct systems: Common JS (JS), which is traditional default, and ES modules (ESM), the modern javascript standard.

1. The common JS method (default):

Common JS uses module.exports to share code and require() to import it. This system loads modules synchornously.

a. Create the module (math.js):

```
const add = (a,b) => a+b;
const subtract = (a,b) => a-b;
module.exports = {
    add
    subtract
};
```

b. use the module (app.js);

```
const math = require('./math');
console.log (math.add (10,5));
console.log (math.subtract (10, 5));
```

O/P: 15
8

2. The ES modules method (modern standard)

ES modules use export to share code and import to use it. To make node.js recognize ES modules, you must either use the .mjs file extension or add "type": "module" to your package.json file.

a. Create the model (Calculate.js):

```
export const multiply = (a,b) => a*b
export const divide = (a,b) => a/b
const calculatorName = "supercale";
export default calculatorName;
```

b. use the modele (main.js)

```
import calculatorName : {multiply, divide}
        from './Calculator.js';

Console.log (calculatorName);
Console.log (multiply (4,2));
Console.log (divide (6,2));
```

O/P: supercale
8
2

Experiment-06

6) Develop mini application using express and node js.

- Simple mini application using node JS + Express;

Student Management System

1. Add a student
2. view all students
3. delete a student
4. uses Express.js Routes
5. simple HTML interface

- Project Structure:

```
Student.app/
  ├ server.js
  └ Public
      └ index.html
```

server.js:

```
Const express = require("express");
Const app = express();
Const PORT = 3000;
let student = [];
app.use(express.json());
app.use(express.static("public"));
app.get("/students", (req, res) => {
    res.json(students);
});
```

```
//Add
app.post("/students", (req, res) => {
    Const student = {
        id: students.length + 1,
        Name: req.body.name,
        Course: req.body.course,
    };
    students.push(student);
    res.json({ message: "student added
                        sucessfully",
               student: student });
});
```

```
// Delete
app.delete("/students/:id", (req, res) => {
    Const id = parseInt(req.params.id);
    Students = students.filter((student) =>
                student.id != id);
    res.json({
        message: "Student deleted successfully"
    });
```

```
//start server
app.listen(PORT, () => {
    Console.log(`server running at http://
    localhost:${PORT}`);
});
```

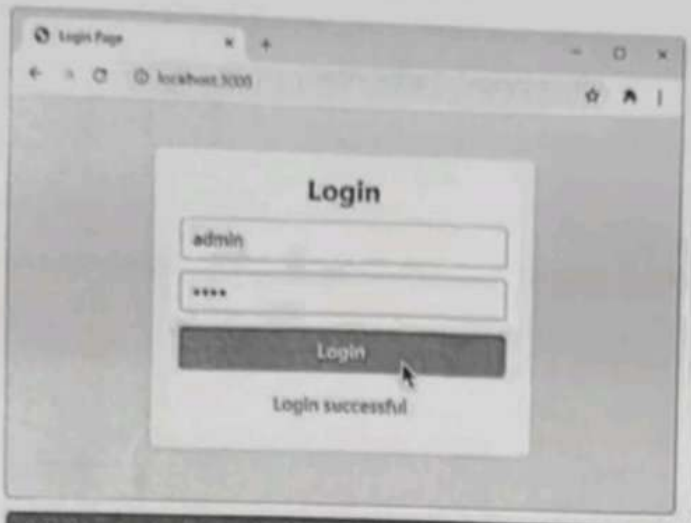
Public/index.html

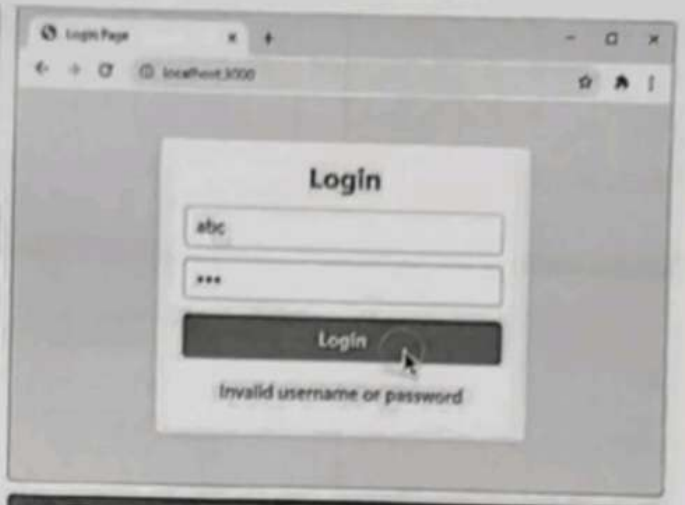
```
<!DOCTYPE html>
<html>
<head>
  <title> Student Management </title>
</head>
<body>
  <h1> Student Management System </h1>
  <input type="text" id="name"
  placeholder="Enter student name">
  <input type="text" id="course"
  placeholder="Enter Cource">
  <button onclick="addStudent()"> Add Student
  </button>

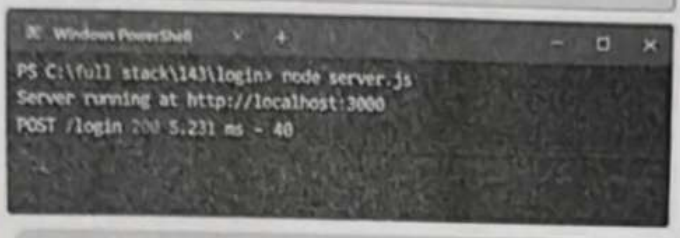
  <h2> Student list </h2>
  <ul id="studentlist"></ul>

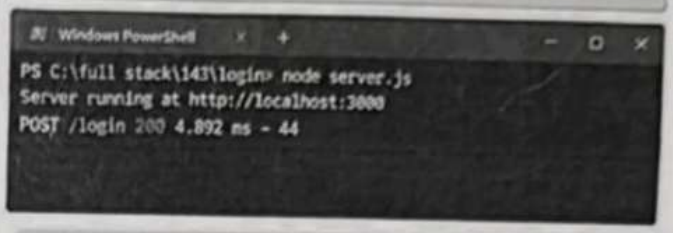
<script>
  function addStudent(){
  let name=document.getElementById("name").
                                    value;
  let course=document.getElement+ById("course")
                                    value;
```

```
fetch("/students", {
    method: "POST",
    headers: {
      "Content-Type": "application/json"
    },
    body: JSON.stringify({
      name: name,
      course: course
    })
  })
  .then(response => response.json())
  .then(data => { alert(data.message);
document.getElementById("name").value = "";
document.getElementById("course").value = "";
    displayStudents();
  });
}
function displayStudents(){
  fetch("/students").then(response => response.json())
  .then(students => {
    let list = document.getElementById
                ("studentlist");
    list.innerHTML = "";
students.forEach(student => { let li =
  document.createElement("li");
```







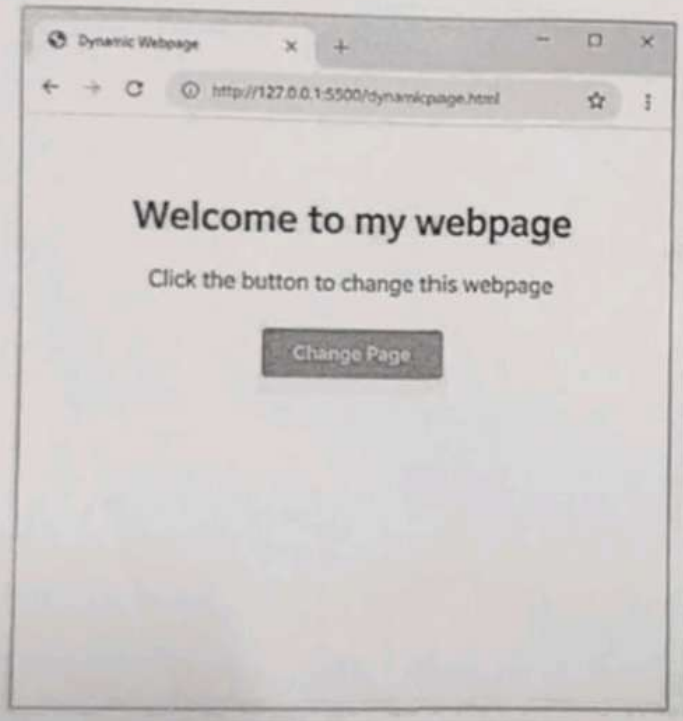


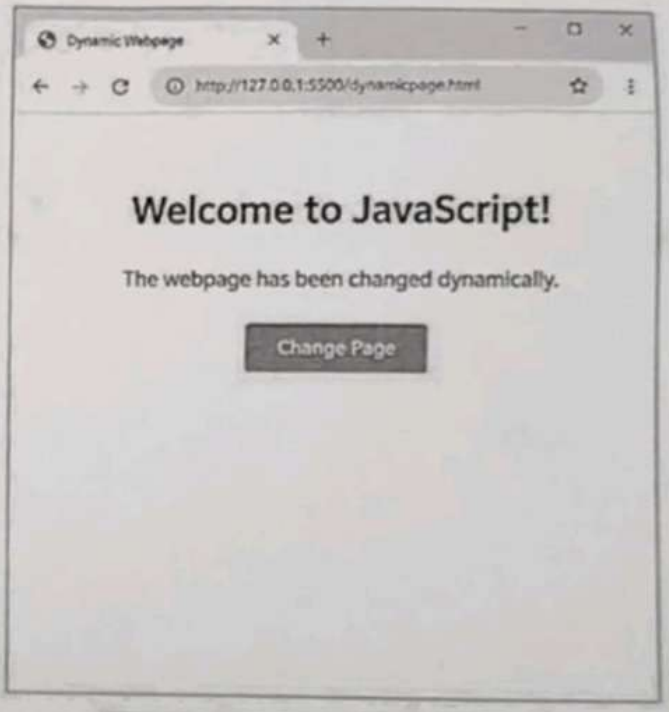
Case 1: Successful Login
(Username: admin | Password: 1234)

Cursor is on the Login button (hover state).

Case 2: Invalid Login
(Username: abc | Password: 123)

Cursor is clicking the Login button (clicked state).





Before clicking the button

After clicking the button

```
li.innerHTML = student.id + " - " +
                student.name + " - " +
                student.course + " <button
onclick = 'deleteStudent(" + student.id + ")'> Delete
</button>";
list.appendChild(li);
        });
    });
}

function deletion(id) {
    fetch ("/students/" + id, {method:
                "DELETE" })
    .then (response) response.json())
    .then (data) {
            alert (data.message);
            displayStudent();
        });
}
displayStudents();

</script>
</body>
</html>
```

Ok.

Student management system

Enter student name

Enter course

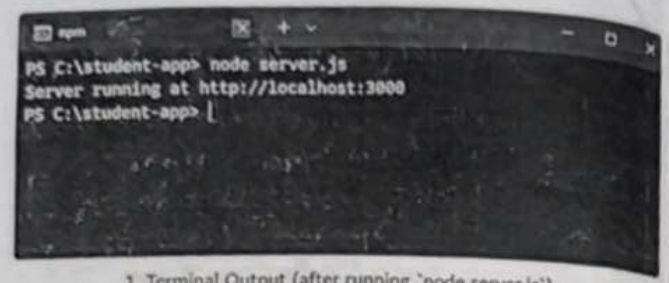
Add student

Student list:

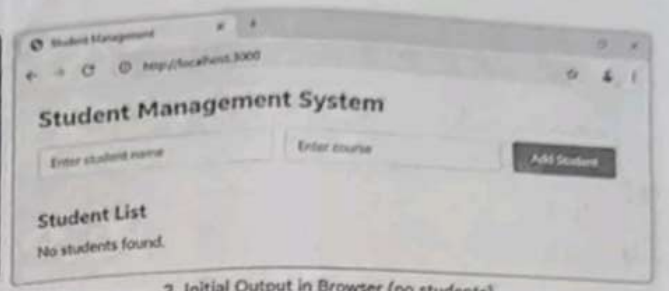
1. Harshu - CSM delete
2. Seenu - CSM delete.

-The express.js server is successfully started at http://localhost:3000

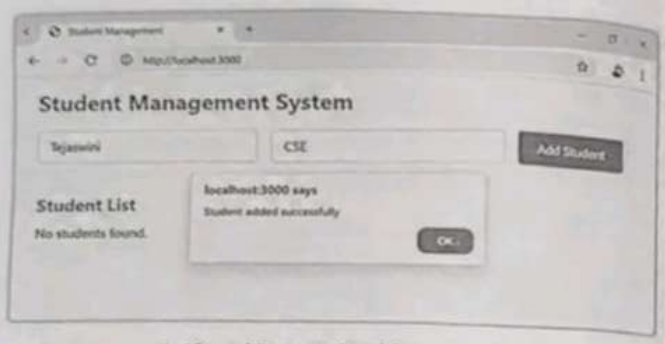
The browser displays the student management system with fields



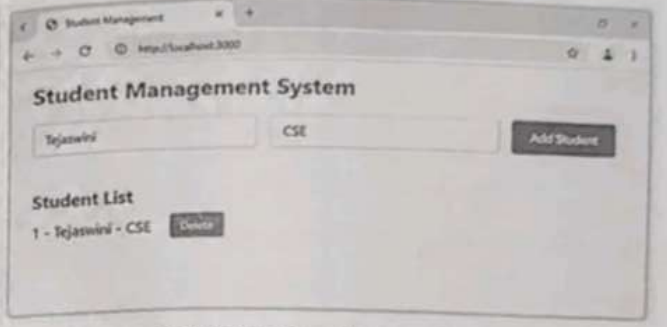
1. Terminal Output (after running `node server.js`)



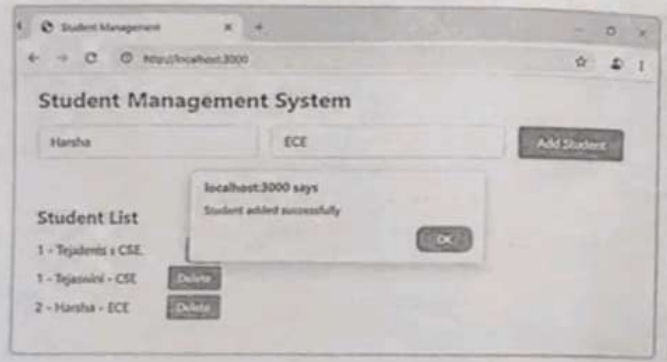
2. Initial Output in Browser (no students)



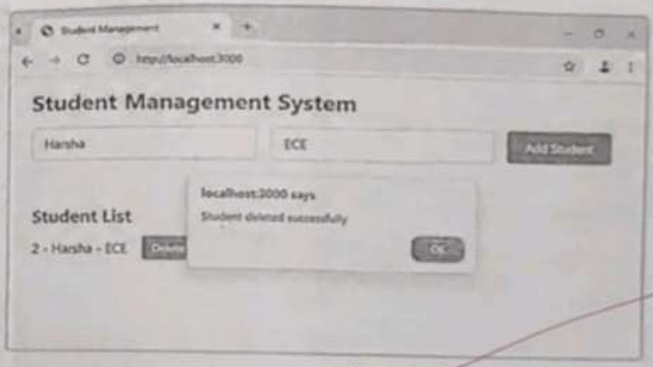
3. After adding a student (alert message)



4. Student list after adding one student



5. After adding another student (alert message)



6. After deleting a student (alert message and updated list)

EXPERIMENT-7

Create a functional component and class component.

Aim: To develop a react application using functional components and class components to display student details, manage state, handle button clicks and dynamically update data on a webpage.

- Functional components:

A functional components is a javascript function that returns JSX to display the user interface. It uses React flooks such as usestate to manage state and update the webpage dynamically.

PROGRAM:

```
import React, {use State} from
'react';
function - Student Functional (){
        const [name, setName] = usestate ("Jas wants");
        const [course, setCourse] = usestate ("CSE");
        const [count, setCount] = useState(0);
        const changeName = () => {
                setName ("Seenu");
        };
const changeCourse = () => {
        setCourse ("CSE");
};
const increment Count = () => {
        setCount (count + 1)
};
return (
    <div style={{
        border : "1px solid blue",
        padding : "20px",
        margin : "10px",
        textAlign: "center",
        borderRadius: "10px"
    }}>
    <h1> Student Details </h1>
    <h2> Functional Component </h2>
    <p> <b>Name: </b>{name} </p>
    <p> <b>Course: </b>{course} </p>
    <p> <b>Button clicks: </b> {count} </p>
    <button onClick = {changeName}>
            Change Name </button>
    <button onClick= {changeCourse}>
            Change Course </button>
    <button onClick= {increment Count}>
            Increment Count </button>
    </div>
);
```

O/P:

When the user clicks the button, the name course and count change dynamic without reloading the webpage.

EXPERIMENT-7

Create a functional component and class component.

Aim: To develop a react application using functional components and class components to display student details, manage state, handle button clicks and dynamically update data on a webpage.

- Functional components:

A functional components is a JavaScript function that returns JSX to display the user interface. It uses React hooks such as usestate to manage state and update the webpage dynamically.

PROGRAM

```
import React, {use State} from
'react';
function - Student Functional (){
        const [name, setName] = useState("Jas warath");
        const [course, setCourse] = useState("");
        const [count, setCount] = useState(0);
        const changeName = () => {
                setName("Seenu");
        };
        const changeCourse = () => {
                setCourse("CSE");
        };
        const increment Count = () => {
                setCount(count+1)
        };
```

```
return (
    <div style={{
        border: "1px solid blue",
        padding: "20px",
        margin: "10px",
        textAlign: "center",
        borderRadius: "10px"
    }}>
    <h1> Student Details</h1>
    <h2> Functional Component </h2>
    <p> <b>Name: </b>{name} </p>
    <p> <b>Course: </b>{course} </p>
    <p> <b> Button Clicks: </b> {count} </p>
    <button onClick = {changeName}>
            Change Name </button>
    <button onClick= {changeCourse}>
            Change Course </button>
    <button onClick= {increment Count}>
            Increment Count </button>
    </div>
);
```

O/P:

When the user clicks the button, the name course and count change dynamic without reloading the webpage.

• Class Component:

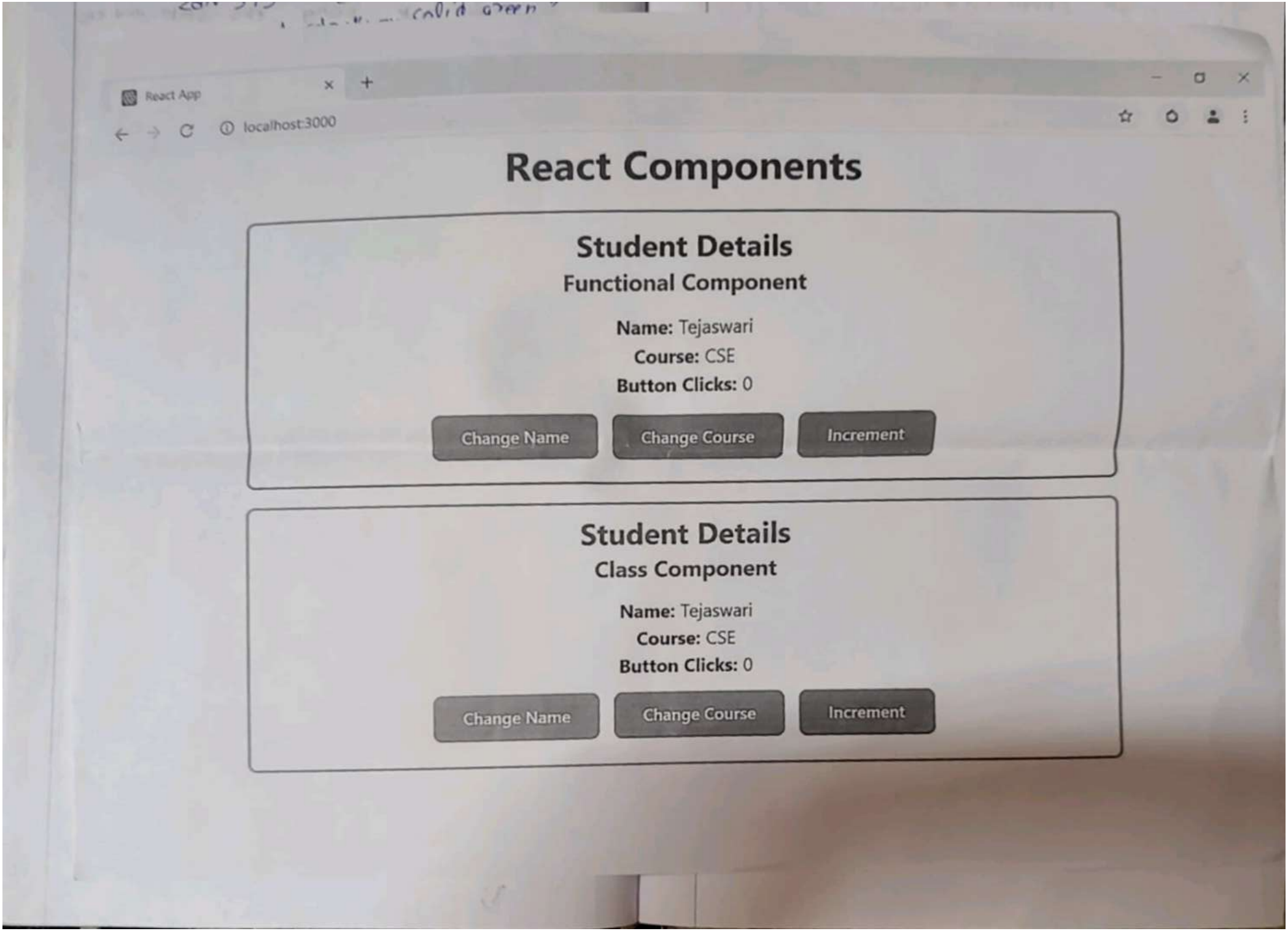
A class component is an ES6 JavaScript class that extends React.Component. It uses the render() method to return JSX and manages state using this.state and this.setState()

program:

```
import react,{Component} from
'react';
class studentclass extends Component{
    constructor (props){
        super(props).
        this.state ={
            name : "Jasvanth";
            course: "CSE"
            count : 0
        };
    }
    changeName = () =>{
        this.setstate ({
            name :"sumitha"
        });
    };
    changecourse= () =>{
        this.setstate ({
            course :"CSE"
        });
    };
    increment count = () =>{
        this.setstate (prevstate) => ({
            count= prevstate.count+1
        }));
};
```

```
render () {
    return (
    <div style={{
            border: "2px solid green",
            padding: "2px",
            margin: "20px",
        borderRadius: "10px",
        textAlign: "center"
    }}>
        <h1>Student Details</h1>
        <h2>Class Component</h2>

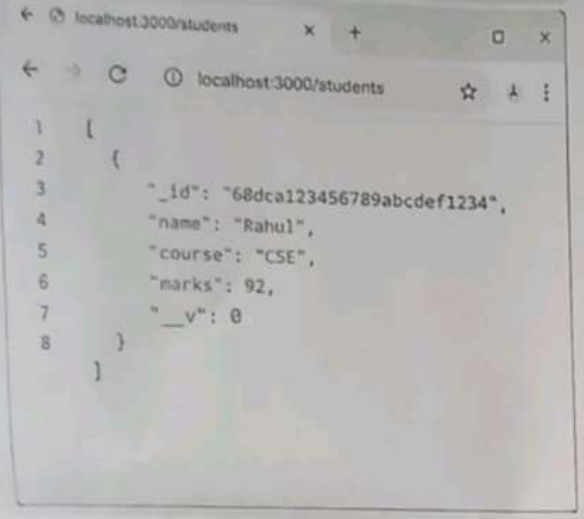
<p><b>Name:</b>{this.state.name}</p>
<p><b>Course:</b>{this.state.course}</p>
<p><b>Button Clicks:</b>{this.state.course}</p>
<button onClick={this.changeName}>
        Change Name</button>
<button onClick={this.changeCourse}>
        Change Course</button>
<button onClick={this.incrementCount}>
        Increment</button>
</DIV>
);
};
```


React App
localhost:3000
React Components
Student Details
Functional Component
Name: Tejaswari
Course: CSE
Button Clicks: 0
Change Name
Change Course
Increment
Student Details
Class Component
Name: Tejaswari
Course: CSE
Button Clicks: 0
Change Name
Change Course
Increment

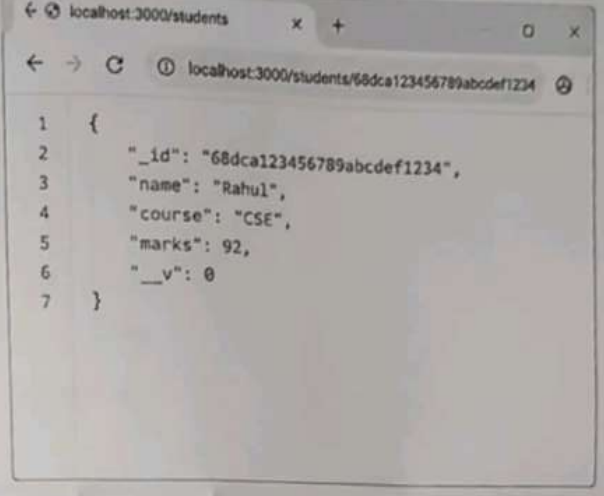
connection using mongoDB

.catch(error => console.log(error));

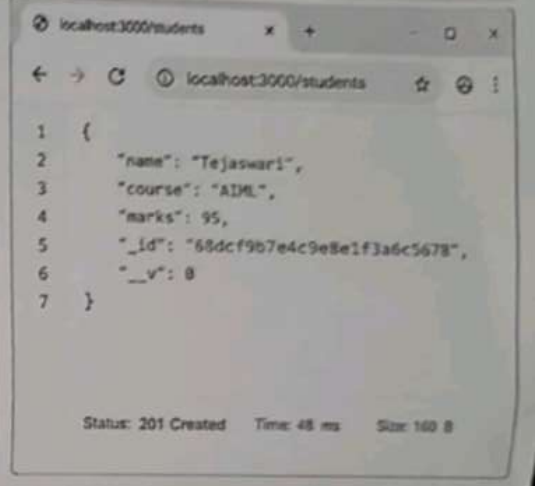
1. GET All Students (http://localhost:3000/students)



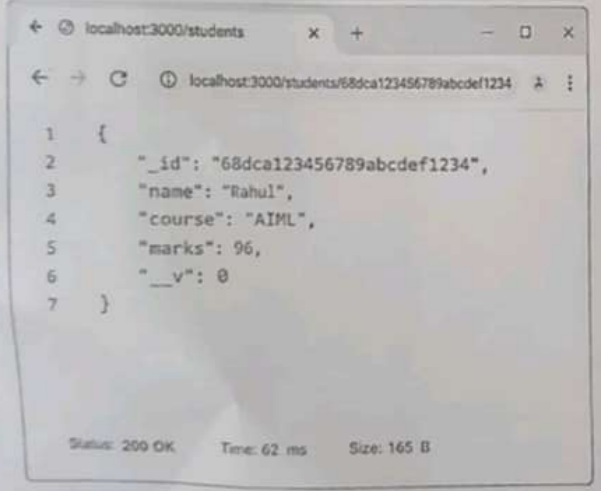
2. GET Student by ID (http://localhost:3000/students/68dca123456789abcdef1234)



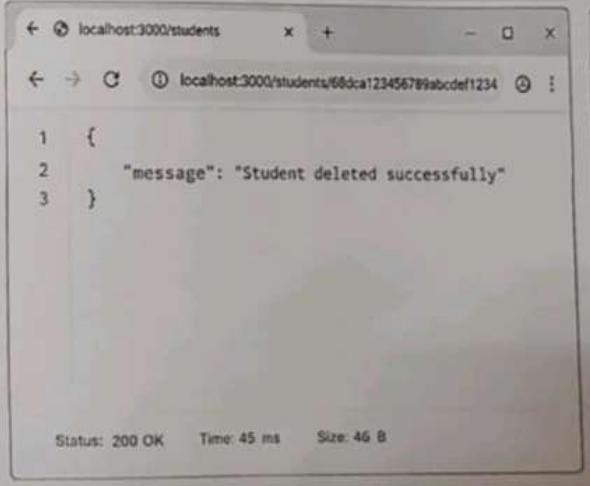
3. POST (Create Student) – Using Postman



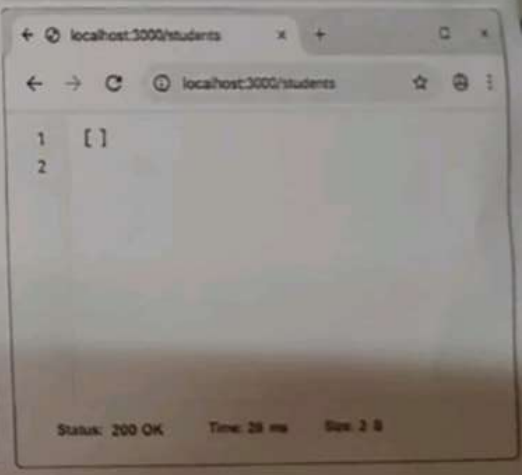
4. PUT (Update Student) – Using Postman



5. DELETE (Remove Student) – Using Postman



6. GET All Students (After Deletion)



app.use(express.json());

mongoose.connect('mongodb://127.0.0.1:27017/college')

Output:

5. DELETE Method:

The delete method is used to remove to existing student record from the server

Code:

```
fetch ('http://localhost:3000/students/3',{
        method: 'DELETE',
    })
then (response => {
    if (response.ok){
        console.log ("student deleted
                    successfully");
    }
})
.catch (error => {
        console.log (error)
});
```

output:

- After executing the POST, GET, PATCH and DELETE operation in sequence the remaining student will be.

output:

```
[
  {
    "id": 1,
    "name": "Ram",
    "age": 20,
    "course": "Computer Science"
  },
  {
    "id": 2,
    "name": "Sita",
    "age": 22,
    "course": "Mathematics"
  }
]
```

Thus HTTP methods were successfully implemented using JSON server the CRUD operation were performed on student records was retrieved, inserted, updated and deleted using HTTP requests.

C. PUT METHOD:

The PUT method is used to replace an existing student record with updated data

Code:

```
fetch ("http://localhost:3000/students/1",{
    method:'PUT',
    header:{
      'Content-TYPE': 'application\s'son',
    },
    body: JSON.Stringify({
        name:'sunitha',
        course:'AIML',
        marks :95
    })
})
.then(response => response.json())
.the (data =>{
    console.log(data);
})
.latch (error =>{
    Console.log(error);
});
```

2/1/

D. PATCH Method:

The PATCH method is used to update only selected fields of an existing student record.

Code:

```
fetch(`http://localhost:3000/students/1`,{
    method: 'PATCH',
    header:{
      'Content-TYPE': 'application/json
    },
    body.JSON-stringify({
        marks: 96
    })
})
.then(response) response.json())
.then (data =>{
    console.log(data);
})
.(atch(error) =>{
    Console.log(error);
});
```

2) Implementation of HTTP methods:

a. GET METHOD: The GET METHOD is used to retrieve student records from JSON server.

Code:

```
fetch ("http://localhost:3000/students")
  .then (response => response.json())
  .then (data => {
      console.log (data);
  })
  .catch (error => {
      console.log (error);
  });
```

output:

b) post method: The post method is used to insert a new student record into the server.

Code:

```
fetch ("http://localhost:3000/students", {
  method: 'POST',
  headers: {
    'Content-TYPE': 'application/json'
  },
  body: JSON.stringify ({
      name: "Rahul",
      course: 'CSE',
      Marks: 91
  })
})
.then (response => response.json())
.then (data => {
    console.log (data);
})
.catch (error => {
    console.log (error);
});
```

O/P:

EXPERIMENT-8

Implement http method using JSON server

Aim:
To implement HTTP method such as GET, POST, PUT and using JSON server to perform CRUD operation on student records

THEORY:

HTTP methods are used to communicate between client and server.

- GET - Retrieves data from the server
- POST - Adds new data to the server
- PUT - Replace an existing record.
- PATCH - Update selected fields of a record.
- DELETE - Remove a record from the server.

a. Create a JSON server:

Step-1 Initialize the project

Open the vs code terminal and execute the following commands

```
mkdir JSONServer
cd JSONServer
npm init -y
npm install JSON Server
```

Step 2: Create a db.JSON ~~Server~~

Create a file named db.json and add the following student data

```
{
  "students":[
    {
      "id":"1",
      "name":"Jaswanth",
      "course":"CSM",
      "marks":"90"
    },
    {
      "id":"2"
      "name":"Sunil"
      "course":"CSM",
      "marks":"91"
    },
    {
      "id":"3"
      "name":"Jeevan"
      "course":"CSE"
      "marks":"89"
    }
  ]
```

Step3:
Start the JSON Server
For the current JSON server version,
Execute:

```
npx json-server db.json
```

The server start at:
http://localhost:3000
The student records can be accessed via
http://localhost:3000/students.

EXPERIMENT-4

Develop CRUD operation using MongoDB

Aim: To develop a web application using Node.js, express.js and MongoDB to perform CRUD operation (Create, Read, update, delete) on student records.

THEORY:

MongoDB is a NoSQL database that stores data in flexible JSON like documents. CRUD operations are used to manage data in a database.

1. Installation and project setup:

```
mkdir MongoDB.CRUD
cd MongoDB.CRUD
npm init -y
npm install express mongoose.
```

PROGRAM:

server.js

```
const express = require ('express');
const mongoose = require ('mongoose');
const app = express();
const port = 3000;
app.use (express.json());
mongoose.connect ('mongodb://127.0.0.1:
                   27017/college)
.then (() => console.log ("MongoDB Connected"))
.catch (error => console.log (error));

//create student
const studentSchema = new
mongoose.Schema({
    name: {
    type: String
    required: true
    }
});
//create student model
const Student =
    mongoose.model ("Student", studentSchema)
// Create API
app.post ('/students', async (req, res) => {
    try {
    const student = new Student (req.body)
    const savedStudent = await student.save();

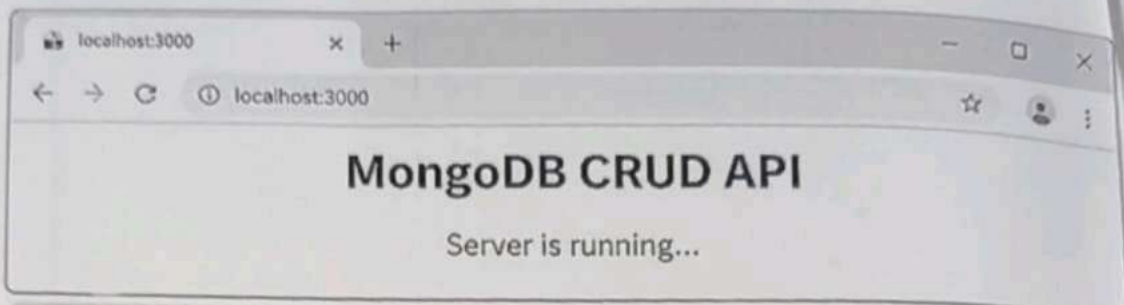
res.status (201).json (savedStudent);
} catch (error){
res.status (400).json ({message: error.message})
    }
});
//READ. Get all students
app.get ('/students', async (req, res) => {
    try {
    const students = await
    Student.find();
    res.status (500).json ({ message: error
                            message});
    }
});
```

```
//Read GET student by ID
app.get('/students/:id', async (req, res) => {
  try {
    const student = await
    Student.findById(req.params.id);
    if (!student) {
      return res.status(404).json({
        message: "student not found"
      });
    }
    res.status(200).json(student)
  } catch (error){
    res.status(400).json({message:
      error.message});
  }
});

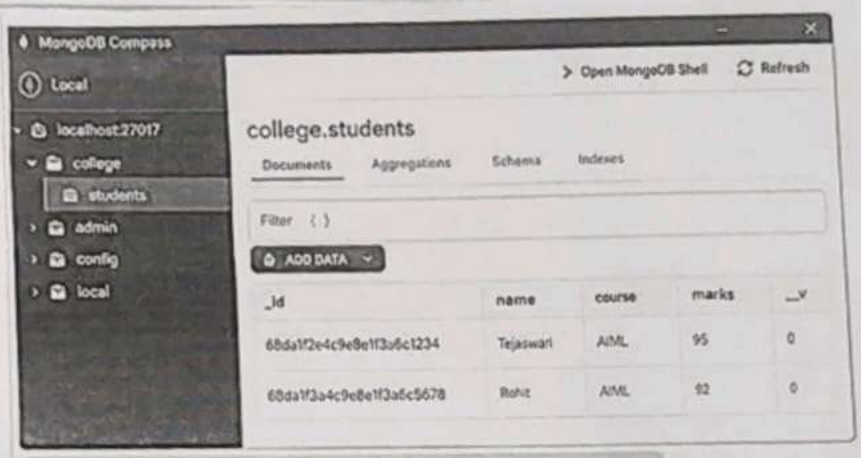
//update
app.put('/student/:id', async (req, res) => {
  try {
    const student = await
    Student.findByIdAndUpdate(
      req.params.id
      req.body
      {new: true, runValidators: true}
    );
    if (!student) {
      return res.status(404).json({
        message: "student not found"
      });
    }
    res.status(200).json(student);
  } catch (error){
    res.status(400).json({message:
      error.message});
  }
});

//DELETE
app.delete('/students/:id', async (req, res) => {
  try {
    const student = await student.
    findByIdAndDELETE(
      req.params.id
    );
    if (!student) {
      return res.status(404).json({
        message: "student not found"
      });
    }
    res.status(200).json({
      message: "student deleted successfully"
    });
  } catch (error){
    res.status(400).json({message:
      error.message});
  }
});
```

Output:-



2. Chrome Browser (Home Page)



8. MongoDB Compass – Database View



1. Terminal Output